

C and Software Tools Exam 2

Name _____

Unity/EOS ID _____

- This test is worth 100 points. There are actually 106 points available. If you score more than 100, you've earned yourself some extra credit.
- This test consists of 8 pages. Make sure your copy has them all.
- Use the space provided for your answer to each problem. If you run out of room, you can continue on to the back of the page (or the back of the previous page). Be sure to indicate this at the bottom of the answer space, so that parts of your answer aren't overlooked.
- Read each problem carefully. You don't want to lose points just because you overlooked part of a question.
- In general, you don't need to comment the code you write or follow our course style guidelines.
- This exam is closed book and closed notes. If you've prepared a handwritten 3×5 card of notes, you may use it during the exam, but you must turn it in at the end. You may **not** use a calculator or any other device in completing this exam. Your work must be individual, and cheating will be punished according to university academic integrity code.

On my honor: I affirm that I have not and will not use any sources of assistance (notes, another student's test, etc.) that are not explicitly authorized during this exam. I am the student whose name appears on this exam.

Signed: _____

Date _____

1. Function Pointer and Array Declarations

- (a) Use typedef to define a type named `WordScore`, representing a pointer to a function that takes a string and returns a double. Indicate that the function won't change the contents of the string. (3 pts)

```
typedef double (*WordScore)( char const *str );
```

- (b) Using this new type name, declare an array, `ptrList`, with room for 25 function pointer of this type. (2 pts)

```
WordScore ptrList[ 25 ];
```

- (c) Using this new type name, create a dynamically-allocated array, `funcList`, with room for 1,000,000 function pointer of this type. (2 pts)

```
WordScore *funcList = (WordScore *) malloc( 1000000 * sizeof( WordScore ) );
```

- (d) **Without** using the `WordScore` type name, create an array, `functions`, with room for 15 function pointer of the same type as `WordScore`. (2 pts)

```
double (*functions[ 15 ])( char const *str );
```

- (e) Pretend `ptrList` has been filled with function pointers. Give a fragment of code to call the last pointer on this list, passing in the string, "abc" and storing the result in a variable named `r`. (2 pts)

```
double r = ptrList[ 24 ]( "abc" );
```

2. **Nested Structs and Arrays**, Use this definition of `Point` for the following questions.

```
struct Point {  
    int coord[ 2 ];  
    double len;  
};
```

- (a) Declare a variable named `pt` storing an instance of `Point`. Use initialization syntax to set the elements of the `coord` array in `pt` to 3 and 4 and set `len` to 5. (3 pts)

```
struct Point pt = { { 3, 4 }, 5 };
```

- (b) Create an array of 3 point instances named `points`. Use partial initialization syntax to set the `len` field of the first array element to 6.32. Don't explicitly initialize anything else. (2 pts)

```
struct Point points[ 3 ] = {  
    { .len = 6.32 }  
};
```

- (c) In the first element of `points`, set the last element of the `coord` array to 6. Do this with an assignment (not initialization). (2 pts)

```
points[ 0 ].coord[ 1 ] = 6;
```

- (d) Using as little code as possible, copy the contents of `pt` to the last element of `points`. (2 pts)

```
points[ 2 ] = pt;
```

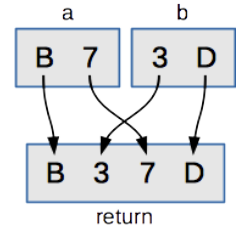
- (e) At minimum, how many bytes of memory might the `points` array use on the common platform? (2 pts)

48 bytes

3. Bitwise operators (4 pts each)

- (a) Fill in the body of the following function so that it returns a 16-bit value based on the two 8-bit parameters. As illustrated in the figure to the right, the 4 high-order bits of the return value should be a copy of the four high-order bits from a. The next 4 bits of the return should be the 4 high-order bits from b. The next 4 bits should be the 4 low-order bits from a, and the last 4 bits should be the 4 low-order bits from b. For example, if a is 0xB7 and b is 0x3D, the return will be 0xB37D.

```
unsigned short interleave( unsigned char a, unsigned char b )
{
    return ( a & 0xF0 ) << 8 |
           ( b & 0xF0 ) << 4 |
           ( a & 0x0F ) << 4 |
           ( b & 0x0F );
}
```



- (b) The following function is supposed to examine the even numbered bits in the given value, bit 0 (the lowest order bit), bit 2, and so on. This function should return 2 if all these bits are set to 1 in val. It should return 1 if any of these bits are set to 1 (but not all of them). It should return 0 otherwise.

```
int testEven( unsigned short val )
{
    if ( ( val & 0x5555 ) == 0x5555 )
        return 2;
    if ( ( val & 0x5555 ) != 0 )
        return 1;
    return 0;
}
```

4. Data Structures

- (a) Give a definition for struct, **Node**, that could be used to build a linked list. Each node should be able to store an integer and a string that's at most 20 characters in length. Use typedef to define **Node** as a name for this type of struct. (5 pts)

```
struct NodeStruct {
    int val;
    char str[ 21 ];
    struct NodeStruct *next;
};

typedef struct NodeStruct Node;
```

- (b) Fill in the body of this function so that it creates a new node containing the given values and inserts it at the front of a linked list. Here, the head pointer for the list is passed in by reference. (5 pts)

```
void insert( Node **head, int val, char *str )
{
    Node *n = (Node *) malloc( sizeof( Node ) );
    n->val = val;
    strcpy( n->str, str );

    n->next = *head;
    *head = n;
}
```

5. Dynamic Memory Allocation

- (a) I've left the body of the following function unspecified. Even without seeing all the code, you can tell that this function is likely to crash when you call it. Comment out the list declaration and replace it with a dynamically allocated array that the rest of the function can use just like the original list. Be sure not to leak memory. (6 pts)

```
void f( int x )
{
    // int list[ 10000000 ];
    int *list = (int *) malloc( 10000000 * sizeof( int ) );

    assert( list ); // for part b.

    ... code using list ...

    free( list );
}
```

- (b) Add an assert to your code above to make sure the dynamic memory allocation is successful. (2 pts)
6. Fill in the following function so that it copies the contents of the string **b**, to **a**. Just use the **a** and **b** pointers; don't declare any other variables inside the function. As you copy characters, omit anything that the C standard library would classify as whitespace. For example, if "This is a test" was passed in as **b**, then **a** would be set to contain "Thisisatest". (8 pts)

```
void squeeze( char *a, char *b )
{
    while ( *b ) {
        if ( !isspace( *b ) )
            *(a++) = *b;
        b++;
    }
    *a = '\0';
}
```

7. Consider the following decimal value. If this is converted to an IEEE single-precision floating point value (a float on the common platform), what sequence of bits will be stored in the mantissa part of the representation. The following figure has many of the low-order bits of the mantissa already filled in for you. You just have to show what will go in the high-order part of the representation. (3 pts)

18.5

0	0	1	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---	---

8. **Fill-in the Blank**, For each of the following, write in the best word(s), term(s) or bits of C syntax to complete the statement. (2 pts each)
- (a) When you call `fopen()`, the letter 'b' in the mode string tells the standard library to open the file in binary mode;
 - (b) The lifetime of dynamically allocated memory is from the time it's allocated until the program calls free.
 - (c) The compiler may leave padding between the fields of a struct in order to satisfy alignment requirements of the hardware or to help speed execution.
 - (d) It's an error to use a pointer that no longer points to dynamically allocated memory. This type of error is called a(n) dangling pointer.
 - (e) The function in question 5a will crash because it exceeds the capacity of the stack.
 - (f) For the code added for question 5b to work, you would need to include the assert.h header.
 - (g) If a C program takes command-line arguments, then `main()` needs two parameters. These parameters are usually named argc and argv.
 - (h) In class, we covered two functions in the C standard library that take function pointers as parameters. One of them was `atexit()` and the other was qsort.
 - (i) The `perror()` function prints out an error message based on the value of the errno variable.

9. **Multiple Choice**, Mark the best answer for each of the following questions. (3 pts each)

- (a) Pretend you have an array declared as follows. Which pointer type can hold the address `a` evaluates to?

```
double a[ 5 ][ 10 ][ 15 ];
```

- ☐ `double *p = a;`
 - ☒ `double (*p)[ 10 ][ 15 ] = a;`
 - ☐ `double *p[ 10 ][ 5 ] = a;`
 - ☐ `double *(*p[ 15 ]) = a;`
- (b) What's the difference between `malloc()` and `realloc()`?
- ☐ `malloc()` allocates memory from the heap, but `realloc()` returns statically allocated memory.
 - ☐ a program request blocks of any size with `malloc()`, but `realloc()` only allocates blocks of a compiler-defined size.
 - ☒ `malloc()` allocates a new block of memory, but `realloc()` exchanges a block for a larger or smaller size.
 - ☐ `malloc()` is implemented as a function, while `realloc()` is an operator, implemented by the compiler
- (c) If `p` is a pointer holding the address `0x12B34520`, what value will `p + 1` have?
- ☐ `0x12B34521` ☐ `0x12B34522` ☐ `0x12B34524`
 - ☒ There's not enough information to answer the question.
- (d) Of the bitwise operators, which one has the highest precedence?
- ☐ `<<` ☐ `>>` ☐ `&` ☐ `|` ☒ `~` ☐ `^`
- (e) After compiling program, `p` with the `-ftest-coverage` and `-fprofile-arcs` options, what do you need to do to run `p` so it records coverage information as it executes?
- ☒ Run `p` normally ☐ Run `p` like: `gcov ./p`
 - ☐ Run `p` like: `gdb ./p` ☐ Run `p` like: `gcc ./p`
- (f) Which of the following is the right way to run a program named `prog` in `valgrind`, with a report of all leaked blocks of memory at the end?
- ☐ `valgrind ./prog --leak-check=all`
 - ☒ `valgrind --leak-check=full ./prog`
 - ☐ `valgrind < ./prog --leak-check=full`
 - ☐ `./prog --valgrind --leak-check=all`
- (g) What is `( 0xFB & 0x7C ) << 1` ?
- ☒ `0xF0` ☐ `0xF8` ☐ `0x78` ☐ `0x70`
- (h) Imagine you have to store a list of 100 strings. If you stored them in a two-dimensional array of characters, every string on the list would use the same amount of storage. Instead, storing the list with an array like the following would

let you vary the amount of memory used to store each string. You could use longer arrays to store longer strings and shorter arrays to store shorter ones.

- ☐ `char strings[ 100 ][];`
- ☐ `char (*strings)[ 100 ];`
- ☐ `char (strings[]) [ 100 ];`
- ☒ `char *strings[ 100 ];`

(i) What does `scanf()` do when there is a `%n` in the format string?

- ☐ It attempts to read an unsigned integer
- ☒ It reports the number of characters that have been read
- ☐ It ignores the value parsed by the next conversion specifier
- ☐ It parses any sequence of non-whitespace characters and stores them in the next argument.